

FPGA Module Connections & I/O Guide

Main Module: Top_CipherX8

INPUTS (What Goes INTO the Main Module)

verilog

```
input [15:0] sw,           // 16 switches (like 16 on/off buttons)
input btnC,               // 1 center button (press to activate)
input clk                 // 1 clock signal (system heartbeat)
```

Total Inputs: 18 signals

- 16 switches (numbered 0 to 15)
- 1 button
- 1 clock

OUTPUTS (What Comes OUT of the Main Module)

verilog

```
output reg [15:0] led,    // 16 LEDs (lights that can turn on/off)
output wire [3:0] an      // 4 seven-segment display controls (turned off)
```

Total Outputs: 20 signals

- 16 LEDs (numbered 0 to 15)
- 4 seven-segment display anodes (not used, kept off)

How Switches Are Used

The 16 switches are divided into two groups:

Switch Layout:

```
[15][14][13][12][11][10][9][8] | [7][6][5][4][3][2][1][0]
      B (upper 8)                |      A (lower 8)
```

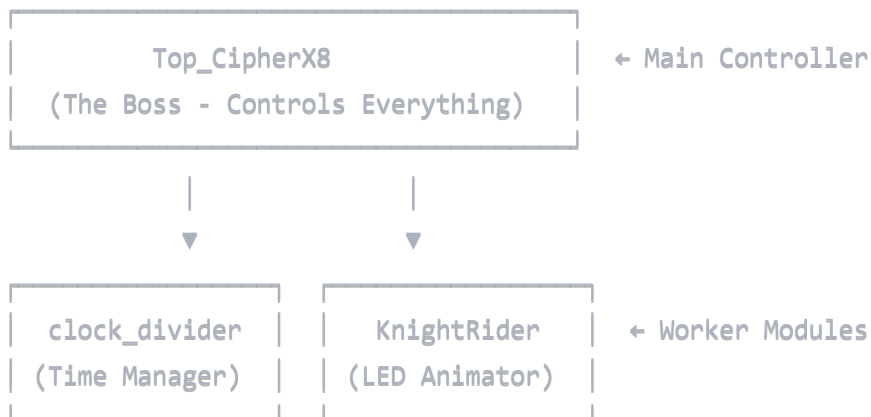
In Code:

verilog

```
wire [7:0] A = sw[7:0];    // A gets switches 0,1,2,3,4,5,6,7
wire [7:0] B = sw[15:8];   // B gets switches 8,9,10,11,12,13,14,15
```

Module Hierarchy & Connections

Think of this like building with LEGO blocks:



Connection Details

Connection 1: Main Module → Clock Divider

```
verilog

// In Top_CipherX8:
clock_divider pulse_20 (
    .pulse_out(clk8),      // OUTPUT: Slow clock signal
    .frequency(8),         // INPUT: Want 8 pulses per second
    .clk_in(clk)           // INPUT: Fast 100MHz clock
);
```

What This Does:

- Main module says: "I need a slow 8Hz clock"
- Clock divider says: "OK, I'll convert 100MHz to 8Hz for you"
- `clk8` wire carries the slow clock signal

Connection 2: Main Module → Knight Rider

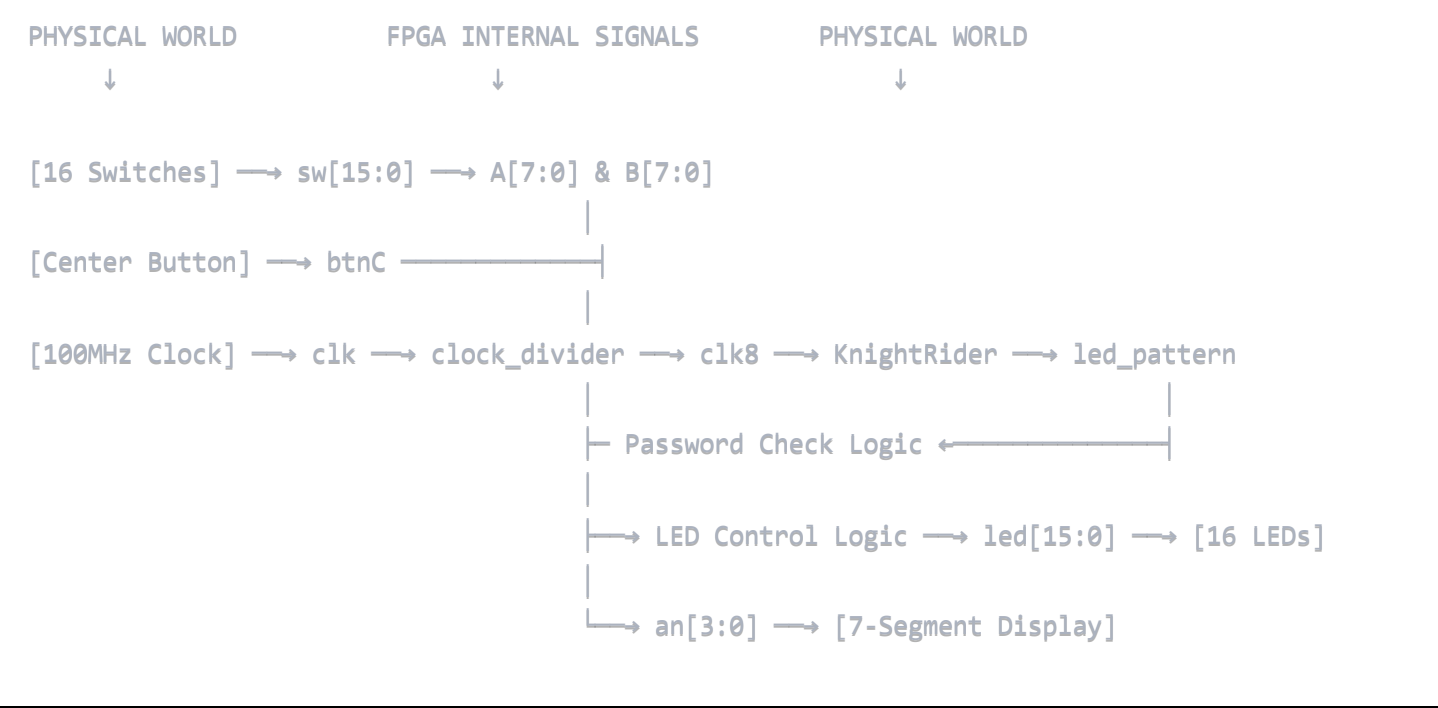
```
verilog

// In Top_CipherX8:
KnightRider knight_rider (
    .led_pattern(led_pattern), // OUTPUT: 16-bit LED pattern
    .clk_KR(clk8)              // INPUT: Slow 8Hz clock
);
```

What This Does:

- Main module says: "Create LED patterns using this slow clock"
- Knight Rider says: "OK, I'll make moving LED patterns"
- `led_pattern` wire carries the LED pattern

Signal Flow Diagram



Step-by-Step Operation

Step 1: Input Processing

```
verilog

wire [7:0] A = sw[7:0];           // Get number A from switches 0-7
wire [7:0] B = sw[15:8];         // Get number B from switches 8-15
```

Step 2: Clock Generation

```
verilog

clock_divider pulse_20 (.pulse_out(clk8), .frequency(8), .clk_in(clk));
//                               ↑           ↑           ↑
//                               slow clock out 8 times/sec fast clock in
```

Step 3: Pattern Generation

verilog

```
KnightRider knight_rider (.led_pattern(led_pattern), .clk_KR(clk8));  
//                               ↑                               ↑  
//                               moving pattern out             slow clock in
```

Step 4: Main Control Logic

verilog

```
always @(posedge clk) begin                                // Every fast clock cycle  
    if (btnC) begin                                       // If button pressed  
        if ((A ^ B) == password) begin                 // If password correct  
            led <= led_pattern;                         // Show moving pattern  
        end else begin  
            led <= 16'b0;                                // ALL LEDs off  
        end  
    end else begin  
        led <= 16'b0;                                    // ALL LEDs off  
    end  
end  
end
```

Module Communication Summary

clock_divider Module

- **Job:** Convert fast clock to slow clock
- **Gets:** 100MHz clock, desired frequency (8)
- **Gives:** 8Hz pulse signal

KnightRider Module

- **Job:** Create moving LED pattern
- **Gets:** 8Hz clock signal
- **Gives:** 16-bit LED pattern

Top_CipherX8 Module (Main)

- **Job:** Coordinate everything, check password, control LEDs
- **Gets:** Switches, button, fast clock, LED pattern from KnightRider
- **Gives:** LED control signals, 7-segment control

Think of It Like a Team

👑 BOSS (Top_CipherX8):
"I need to check passwords and control LEDs"

🕒 TIMER (clock_divider):
"I'll give you perfect timing signals"

🎮 ANIMATOR (KnightRider):
"I'll create cool moving patterns"

BOSS: "Timer, give me slow pulses"
TIMER: "Here's your 8Hz signal"

BOSS: "Animator, make patterns with these pulses"
ANIMATOR: "Here's your moving LED pattern"

BOSS: "Perfect! Now I'll decide when to show it based on the password"

This modular design is exactly like how you structure mobile apps - you have different components that do specific jobs and communicate with each other!